1. We had to instrument our code by including net/http/pprof.
2. We had to include a web server to expose the profiles (alternatively, we could have written out the data to a file).
3. We had to use a browser or `curl` (and other dependencies, such as `dot`) to get a "point in time" result and, hence, an answer to our resource usage question.
4. We had to wait 30 seconds for `pprof` to gather profiles we were then able to query interactively.

NOTE Truth to be told, I've been using Go as an example here, since it comes with a powerful profiler already built into its tool chain and standard library. The same exercise for other programming languages would likely mean even more effort.

For example, here are a few language-specific approaches to collect profiles:

- Java has async-profiler (https://github.com/jvm-profiling-tools/async-profiler)
- .NET has dotTrace (https://www.jetbrains.com/profiler/)
- PHP has phpspy (https://github.com/adsr/phpspy)
- Python has `cProfile` (https://docs.python.org/3/library/profile.html)
- Ruby has `rbspy` (https://rbspy.github.io/)
- Rust has `pprof-rs` (https://github.com/tikv/pprof-rs)

Don't get me wrong; as you can see in the Go example, profiling is possible but requires specialized knowledge, just like using containers before around 2015 (when Docker became available) was doable but required intimate knowledge of Linux kernel features, including cgroups and namespaces. Nowadays, running a container is a mere `docker run` away for anyone.

But then, in 2010, something happened: a bunch of Google engineers published the seminal paper "Google-Wide Profiling: A Continuous Profiling Infrastructure for Data Centers" by Gang Ren, Eric Tune, Tipp Moseley, Yixin Shi, Silvius Rus, and Robert Hundt (2010, IEEE Micro, https://research.google/pubs/pub36575/). In this paper, the authors described the idea of continuously capturing CPU and memory usage and associating this data (the profiles) with the source code so that it enables engineers to locate slow or buggy lines of code as well as shared lessons learned using their continuous profiling system at scale. This paper influenced the community, and in the following years, we witnessed a number of commercial offerings growing in said space. In conclusion, profiling is doable using the primitives, but creating and consuming profiles on that level is complicated and, hence, it didn't go mainstream.

The tradeoff that continuous profiling takes is to profile at relatively low frequency and gather statistical significance over time. This is in contrast to traditional profiling, where we profile at very high frequency because we want to reach statistical significance over a short period of time—say, 10 seconds. Now, before we get into continuous profiling tooling, let us first step back and discuss some common techniques and technologies widely used in this space.